Project Summary: CNN-Based Pulse Counting for Synthetic GRB Light Curves

Condensed summary for discussion. Full technical methodology, all code documentation, and complete results are in `REPORT.md` (also available as `REPORT.pdf`) in this same folder.

Objective

Train a 1D convolutional neural network to predict, directly from a gamma-ray burst light curve, how many pulses it contains — using the existing Stern & Svensson (1996) pulse-avalanche simulator already in this repository as a source of unlimited, exactly-labeled synthetic training data (real Swift/BAT catalog data is not available on this machine and, as explained below, isn't actually needed for this).

What was built

1. **Fixed a pre-existing bug** in the repository that silently broke the light-curve generation script (`simulate_GRBs.py`) — a variable-name collision meant it was ignoring its own configuration file.
2. **Added a new label.** The simulator already computes, internally, the number of "parent" pulses that seed each simulated burst's avalanche — physically, a good proxy for "the number of peaks a human would count in the light curve" — but discarded it after use. This was exposed and wired through to the exported data files.
3. **Built the full ML pipeline** (PyTorch): a data loader, a compact 1D CNN, a training script, and a validation script that reports accuracy and produces diagnostic plots.
4. **Ran a controlled experiment, not just one training run** — a baseline model, then a specific, justified fix for a problem the baseline revealed, retested on the *same* data and split to isolate cause from effect (methodology detailed below).
5. **Documented everything** to a reproducible methods-paper standard: every design decision has a stated rationale, and the full pipeline can be re-run from scratch on another machine using the instructions in `REPORT.md`.

Key finding: the baseline model looked fine, but wasn't

Trained on 3,000 synthetic light curves with a standard setup (plain MSE regression loss, random batch sampling), the model reached:

- 46.2% exact-match accuracy, 92.2% accuracy within ±1 pulse

Those numbers look reasonable — but breaking accuracy down **by pulse count** revealed the model had stopped predicting anything above 2 pulses entirely, regardless of the true count. It had effectively learned to always guess the population mode. This was invisible in the aggregate metrics alone (they looked fine mostly because 77% of the data genuinely does have only 1-2 pulses) and only surfaced once we plotted the true-vs-predicted distribution and checked per-class accuracy directly.

Why: two compounding causes. (1) The physical model's fixed parameters mean high pulse counts are intrinsically rare (6-pulse bursts are 0.3% of the data) — a direct, unavoidable consequence of the calibrated physics, not a data-collection artifact. (2) Standard squared-error regression loss mathematically rewards a model for defaulting to the dominant classes when they vastly outnumber the rare ones.

The fix, tested in isolation

Implemented two targeted, well-established remedies for this specific failure mode — weighted batch sampling (so rare pulse-counts are seen as often as common ones during training) and a count-appropriate loss function (Poisson, matching how the label is actually generated, instead of MSE) — and retrained on the **identical** 3,000-light-curve dataset and train/test split, changing only that.

Result: it worked, partially, and honestly. The model stopped taking the shortcut (3-pulse accuracy went from 0% to 9.6%; 1-pulse accuracy improved substantially), but 4-, 5-, and 6-pulse light curves still saw no improvement — because with only 6-23 examples of those classes in the whole dataset, reweighting them just means replaying the same handful of examples more often, not manufacturing new information to learn from. This isolates the open question precisely: **is this a data-quantity problem, or a harder ceiling (e.g. genuinely overlapping pulses that even a human couldn't distinguish in the noisy signal)?**

Current status — including the 10,000-LC result

- **3,000-LC run:** complete (baseline and fix both trained, compared, documented).
- **10,000-LC run:** complete. Result is genuinely mixed, not a clean "more data fixed it": the 4-pulse class's accuracy roughly doubled (9.6%→18.5%, with 3.5x more test examples of it) — real evidence that scale helps. But the rarest classes (5, 6, 7 pulses) showed **zero** improvement at any dataset size tested so far, since even at 10,000 they only reach 21, 3, and 1 test examples respectively — nowhere near enough yet. Overall aggregate accuracy actually dipped slightly, which the per-class breakdown shows is misleading on its own (full discussion in REPORT.md §7.7).
- **20,000-LC run:** on hold, pending a decision on whether it's worth running given the above — the answer to "does more data resolve this" is "partially yes, for some classes, but 10,000 isn't enough for the rarest ones."

One important caveat, stated plainly

The seven physical parameters driving the simulator **are** the real, published values from fitting this model against the actual Swift/BAT catalog (that calibration work was done previously, by the repository's original authors) — so the *simulated light curves* are statistically realistic. However, the specific label used here (number of parent pulses) is an internal quantity of the generative model with **no directly observable counterpart in real data** — a real GRB has no "true" parent-pulse count to check against. This work demonstrates the CNN can learn the simulator's internal structure; it has not been, and cannot straightforwardly be, validated against real observations for this specific target without additional work (noted as a next step, not done here).

Next steps (open decisions)

1. **Decide whether N=20,000 is worth running.** The 10,000 result is a genuine "partially yes": scale demonstrably helped class 4, but the rarest classes (5-7) need far more data than 10,000 provides before the same effect would show up (estimated 100k+ for class 7 at the current rate of rarity). 20,000 would likely repeat "some improvement, still not enough for the rarest classes" rather than resolve them outright — worth deciding if that's a useful data point or not before spending the ~2.5-3 hours it would take to generate.
2. **Targeted data generation** (varying `mu0` to directly manufacture more high-pulse-count examples, rather than waiting for them to accumulate at their natural rare rate) is the more direct fix for classes 5-7 specifically, if resolving them is the priority.
3. Architecture changes, or accepting the current result as sufficient evidence and moving to a different remedy for the rare-class problem.